

Explication de la Logique du Projet (Gestion Concurrente d'un Aéroport)

1. Cadre Général de la Simulation

Ce projet est une **simulation** du processus de **gestion de la concurrence dans un aéroport unique**. L'objectif principal est de démontrer comment les algorithmes de synchronisation gèrent les **ressources partagées et limitées** face à la compétition de plusieurs entités (les avions).

- **Le Moniteur Principal (L'Aéroport) :** Nous utilisons **un seul objet aéroport** (représenté par l'objet Aeroport ou AeroportSemaphore).
- **Les Acteurs (Threads) :** Les avions sont les acteurs actifs de notre système (les threads). Chaque avion est un processus indépendant qui tente d'accomplir son cycle de vie : atterrissage, stationnement à une porte, et décollage.
 - Nous utilisons des noms de compagnies aériennes clairs (M-Emirates, S-Air France, etc.) pour faciliter le suivi dans les logs.
- **Les Ressources Partagées :**
 1. **Pistes :** Nombre limité (par défaut 2) pour l'atterrissage et le décollage.
 2. **Portes :** Nombre limité (par défaut 4) pour le stationnement et le service passagers.

2. Contraintes Critiques de la Simulation

L'algorithme de synchronisation doit respecter les règles suivantes :

1. **Exclusion Mutuelle :** Une piste ou une porte ne peut être utilisée que par **un seul avion à la fois**.
2. **Priorité :** Les **arrivées (atterrissages) doivent être prioritaires** sur les départs (décollages).
3. **Attente :** Les avions doivent attendre si aucune ressource n'est disponible.

3. Logique de Synchronisation et Comparaison

Nous comparons deux méthodes pour résoudre ce problème de concurrence : les Moniteurs et les Sémaphores.

A. Implémentation par Moniteurs (Aeroport.java)

C'est le mécanisme natif de Java, basé sur le verrouillage et la signalisation.

Élément	Logique d'Implémentation	Justification
Exclusion Mutuelle	Utilisation du mot-clé synchronized sur toutes les méthodes d'accès aux ressources .	Garantit qu'un seul thread (avion) peut exécuter le code critique à la fois.
Attente (Gestion des Files)	Utilisation de la boucle while (condition non remplie) { wait(); }.	Attente Passive : Lorsque le ressource n'est pas disponible, l'avion est mis en sommeil profond (wait()) sans consommer de temps CPU.
Priorité des Départs	L'avion en départ vérifie le double-critère : while (PisteOccupée OU fileArrivées non vide) { wait(); }.	Condition Stricte : L'avion en départ est bloqué jusqu'à ce que la liste d'attente des arrivées soit totalement vide .
Affichage GUI	Les deux listes (fileArrivees et fileDeparts) sont des LinkedList internes au Moniteur.	Clarté : Le nombre d'avions bloqués par la règle de priorité est visible dans la File d'attente Départs de l'interface.

B. Implémentation par Sémaphores (AéroportSemaphore.java)

Utilisation des objets `java.util.concurrent.Semaphore` basés sur un système de permis.

Élément	Logique d'Implémentation	Justification
Exclusion Mutuelle	<code>mutex = new Semaphore(1, true)</code> protège l'accès aux compteurs partagés (<code>nbAttentesArrivees</code> et <code>nbAttentesDeparts</code>).	La mutex garantit que seul un thread à la fois modifie les compteurs ou décide de l'ordre d'accès aux pistes.
Accès aux Pistes	<code>semaphorePistes = new Semaphore(nbPistes, true)</code> et <code>semaphorePistes.tryAcquire()</code> pour tenter d'obtenir une piste	Système de permis : le nombre de pistes limite les accès. Si toutes sont occupées, l'avion est bloqué dans la file d'attente FIFO
Priorité des Arrivées	Les départs ne peuvent acquérir une piste que si <code>nbAttentesArrivees == 0</code> . Sinon, le départ est mis en attente (<code>fileDecollage.acquire()</code>).	Assure que les arrivées ont toujours priorité sur les départs. La logique ne repose pas sur une boucle active mais sur le sémaphore FIFO
Affichage GUI	Les méthodes <code>getNbAttentesArrivees()</code> et <code>getNbAttentesDeparts()</code> permettent d'afficher correctement les files d'attente.	La GUI peut lire les compteurs et les listes de ressources pour montrer les avions en attente avant leur libération par le sémaphore.

4. Explication de la Vue Graphique (GUI)

Lors de votre présentation, utilisez l'interface graphique pour illustrer ces concepts :

➤ **Section "PARAMÈTRES DE LA SIMULATION" :**

- "Cette section permet de modifier le nombre de **Ressources Partagées** (Pistes et Portes) de manière dynamique, puis de réinitialiser le système pour observer l'impact sur la concurrence."

➤ **Section "Mécanisme de Synchronisation" :**

- "Ici, nous choisissons l'algorithme actif : **Moniteurs** (plus efficace en CPU) ou **Sémaphores** (utilisant une boucle d'attente active pour la priorité)."

➤ **Section "STATUT DES RESSOURCES PARTAGÉES" :**

- **Statut des Pistes** : "Montre quelle piste (Piste 1 ou Piste 2) est **LIBRE** ou **OCCUPÉE** par quel avion, prouvant que l'exclusion mutuelle est respectée."
- **Statut des Portes** : "Montre l'état des portes (LIBRE ou OCCUPÉE)."

➤ **Section "FILES D'ATTENTE" (FILES D'ATTENTE) :**

- **File d'attente Arrivées** : "Liste les avions qui attendent une piste pour atterrir."
- **File d'attente Départs** : "Dans le mode **Moniteur**, cette file est la preuve concrète de l'application de la priorité, car elle contient tous les avions bloqués par les arrivées. Dans le mode **Sémaphore**, cette file reste vide car les avions attendent dans leur thread (Thread.sleep())."

➤ **Journal de bord (Logs) :**

- "Ce journal affiche les décisions instantanées du Moniteur (Aéroport M:) et les étapes de chaque avion, confirmant que le critère de priorité est appliqué (ex : 'attend une piste pour décoller (Priorité Arrivées)')."

5. Le Cycle de Vie Aléatoire d'un Avion

5.1. Cas où l'Avion est une ARRIVÉE

Si l'avion est choisi pour être une **Arrivée** (initialState = "Arrivée"), il doit passer par les trois phases complètes de l'aéroport :

Phase	Action	Ressource Demandée	Durée (Max 4s)	Logique de Synchronisation
1. Atterrissage	L'avion entre dans l'espace aérien et demande à se poser.	Piste (Priorité est donnée ici)	Min 2s, Max 4s	Doit attendre si les deux pistes sont occupées.
2. Stationnement	L'avion se rend à la porte pour le service (débarquement/embarquement).	Porte d'embarquement	Min 1s, Max 4s	Doit attendre si toutes les portes sont occupées.
3. Décollage	Le service est terminé, l'avion demande à quitter l'aéroport.	Piste (Soumis à la priorité)	Min 2s, Max 4s	Doit attendre si des arrivées sont en file.

5.2. Cas où l'Avion est un DÉPART

Si l'avion est choisi pour être un **Départ** (initialState = "Départ"), il est supposé être déjà à une porte (en attente de la piste) et ne passe que par la phase finale :

Phase	Action	Ressource Demandée	Durée (Max 4s)	Logique de Synchronisation
1. Décollage	L'avion demande à s'aligner et à décoller.	Piste (Soumis à la priorité)	Min 2s, Max 4s	Doit attendre si des arrivées sont en file ou si les pistes sont occupées.

Résumé de la Logique Aléatoire

Le bouton choisit au hasard entre :

- **Arrivée (Cycle Complet)** : Piste → Porte → Piste
- **Départ (Cycle Court)** : Piste (Seulement le décollage)

C'est pour cela que lorsque vous cliquez, vous voyez parfois l'avion immédiatement demander la piste pour le décollage, et d'autres fois pour l'atterrissage.

6. Comparaison des comportements après simulation

Critère	Moniteurs	Sémaphores
Mécanisme	Attente Passive basée sur une condition (while (!fileArrivees.isEmpty()) { wait(); })	Gestion automatique de la file interne par le sémaphore : acquire() bloque si aucun permis disponible, release() libère un thread dans l'ordre FIFO si true
Comportement	Lorsqu'une piste est libérée, notifyAll() réveille instantanément les threads, qui vérifient la condition. Si la priorité est	L'ordre de libération est automatique et respectueux de FIFO si le sémaphore a été créé avec true. Les arrivées et

	toujours active, les départs retournent immédiatement à l'état wait()	départs sont servis selon les compteurs nbAttentesArrivees et nbAttentesDeparts
Visibilité	La File d'attente Départs se remplit , offrant une preuve visuelle de la mise en file d'attente des threads bloqués.	Les files d'attente gérées par le sémaphore sont internes et invisibles à l'interface graphique. Dans notre code, nous maintenons et affichons nos propres listes pour que l'utilisateur voie l'ordre d'attente.